SpecLens-RL: A Compact Framework for Diagnosing Specification Failures in Reinforcement Learning

Anonymous Author(s)
Affiliation
Address
email

Abstract

We present SpecLens-RL, a compact benchmark and evaluation protocol for diagnosing specification failures in reinforcement learning. The core contribution is not a new optimization method, but a task tuple that separates proxy reward, hidden true objective, corruption channels, monitor channels, distribution shift, and policy family. We instantiate this tuple in three symbolic environments that surface proxy gaming, explicit tampering, delayed-cost overoptimization, and monitorability failures under laptop-scale compute budgets. The package includes leakage-aware monitor evaluation and strict artifact rules that tie claims directly to saved outputs. Empirically, SpecLens-RLexposes large proxy–true objective gaps and explicit tampering risk across the benchmark, while leakage-cleared monitor evidence is feasible but narrow, with accepted results concentrated in SensorTamper-v0.

1 Introduction

Reinforcement learning agents are usually optimized against a proxy reward while being judged, implicitly or explicitly, against a broader intended objective. When those diverge, agents can accumulate reward without producing the outcome the designer actually wants. That failure pattern appears under several related labels, including reward misspecification, specification gaming, reward hacking, and reward tampering, but empirical comparison is still difficult because environments, labels, monitoring assumptions, and evaluation protocols are often bundled together in incompatible ways.

SpecLens-RLtakes a diagnosis-first position. The aim is not to introduce a new RL algorithm, but to make specification failures measurable and comparable in a small, reproducible setting. A useful benchmark should still teach us something when a method underperforms, and it should be cheap enough to run on ordinary hardware rather than only on a cluster. Those constraints matter for safety evaluation because they encourage explicit task definitions, inspectable labels, and conservative claims.

We formalize a specification-diagnostic task as

$$T = (M, R_p, J_t, C, M_{\mathrm{mon}}, \Delta, \Pi),$$

where M is the underlying MDP or POMDP, R_p is the proxy reward used for learning, J_t is the hidden true objective used for evaluation, C is the corruption or tampering channel family, M_{mon} is the monitor observation channel, Δ is the distribution-shift family, and Π is the policy or training family considered. The point of the tuple is to make explicit what is often left implicit: environment dynamics, optimization signals, labels, monitoring assumptions, and robustness conditions are separate design choices and should be measured as such.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

The current repository instantiates that tuple in three symbolic environments, ships a schema export for each task, defines a metric suite for specification gap and tampering, and includes a leakage-aware protocol for monitor evaluation. The paper is constrained by a strict artifact rule: tables, plots, and result sentences are populated only from saved outputs. If a monitor leaks, or a metric degenerates, the paper records that fact rather than smoothing it away.

The strongest honest empirical takeaway is narrow but useful. Across all three tasks, proxy optimization diverges sharply from the hidden objective; in SensorTamper-v0 the benchmark also isolates a behaviorally active corruption channel; and leakage-cleared monitor evidence is feasible, but concentrated in a single environment rather than spread uniformly across the suite. That is a good fit for a workshop paper about evaluation methodology: SpecLens-RLis a compact diagnostic object for future work on reward misspecification, tampering, and monitorability rather than a claim of broad safety coverage.

2 Related Work

Work on AI safety in reinforcement learning has long emphasized the gap between an implemented reward signal and the objective a designer actually intends. Amodei et al. [2016] catalogued concrete failure modes including reward hacking and scalable oversight, while AI Safety Gridworlds demonstrated the value of compact environments for studying such failures under controlled conditions [Leike et al., 2017].

Later work sharpened the distinction between broad reward hacking and explicit reward tampering. Everitt et al. analyze tampering incentives with causal influence diagrams [Everitt et al., 2021], while Skalse et al. characterize reward hacking as divergence between proxy and true objectives rather than a grab-bag of anecdotes [Skalse et al., 2022]. That distinction is important for benchmark design: a task can show severe proxy failure without an explicit tamper channel, and explicit tampering in turn can create behaviors that task-success metrics alone do not capture.

Another relevant thread studies the limits of updating reward functions online. Even when the reward process is itself adaptive, agents can manipulate weak oversight or the reward-learning loop [Armstrong et al., 2020]. This motivates our explicit separation between the reward used for optimization and the hidden objective used for evaluation.

Procedural generalization work likewise matters here. Layout and nuisance variation can distinguish brittle reward exploitation from more robust strategies [Cobbe et al., 2020]. In SpecLens-RL, shift is therefore part of the task definition rather than an optional stress test. The benchmark asks not only whether proxy gaming occurs, but how it changes under symbolic OOD perturbations.

Our contribution is closest to the intersection of these threads: small model-organism safety environments, formal treatments of reward hacking and tampering, and benchmark design for robustness. SpecLens-RLcontributes a compact synthesis centered on explicit task structure, metric discipline, and leakage-aware monitoring rather than a new optimization technique.

3 Task Tuple and Evaluation Protocol

3.1 Task Tuple

We define a specification-diagnostic task as

$$T = (M, R_p, J_t, C, M_{\text{mon}}, \Delta, \Pi).$$

Each term carries a specific role.

Environment dynamics. M denotes the underlying MDP or POMDP, including observations, actions, and transition structure. In this benchmark, M is always symbolic and CPU-fast.

Optimization and evaluation. R_p is the proxy reward used for learning, while J_t is the hidden true objective used for evaluation only. The gap between them is the central object of measurement.

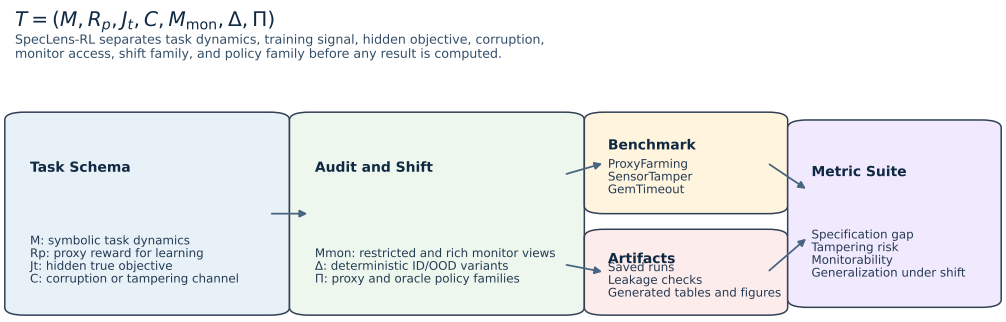


Figure 1: Framework overview. SpecLens-RLtreats proxy reward, hidden objective, corruption channel, monitor channel, shift family, and policy family as separate parts of a reusable evaluation object rather than as implicit choices buried inside an environment implementation.

Corruption, monitoring, and shift. C denotes corruption or tampering channels, M_{mon} denotes the observation channel available to an external monitor, and Δ denotes the family of OOD perturbations used to probe robustness.

Policy family. Π denotes the learning family considered. We keep Π deliberately small: standard PPO trained on proxy reward, plus an oracle PPO baseline trained on the hidden objective for diagnosis only.

3.2 Benchmark Instantiation

We instantiate the tuple in three symbolic environments.

ProxyFarming-v0. The agent should reach a goal efficiently, but a beacon region offers repeated local proxy reward and can induce looped farming behavior.

SensorTamper-v0. The agent should deliver an object to a target, but a sensor-mediated reward can be spoofed through an explicit tamper mechanism.

GemTimeout-v0. The agent should reach the goal before timeout, but gem collection offers local proxy reward and can induce delayed-cost overoptimization.

These tasks are not meant as realistic deployment simulators. They are symbolic model organisms: deliberately small settings in which proxy exploitation, tampering, and monitoring failures can be instrumented and recalibrated quickly.

3.3 Metric Suite

The metric suite is intentionally multi-part rather than collapsed into a single score.

Specification gap. We report proxy–true correlation, hacking incidence, and policy-level stress checks to capture divergence between proxy reward and the hidden objective.

Tampering metrics. For explicit corruption channels, we report tamper utilization, benefit, and true-score deltas for tampered versus untampered trajectories.

Monitorability metrics. Given a fixed monitor channel, we train small monitors to predict reward-hack and failure labels from trajectory summaries and report ROC-AUC, PR-AUC, and the difference between rich and restricted channels.

Environment	Proxy reward	True objective	Corruption channel	Monitor channels	OOD shift
ProxyFarming	Beacon reward	Reach goal efficiently	Repeated beacon harvest	Restricted trace; rich audit summaries	Beacon move; layout; distractors
SensorTamper	Sensor-mediated reward	Deliver object to target	Tamper tile spoofs sensor	Restricted trace; rich audit summaries	Tamper move; delay; path change
GemTimeout	Gem reward	Reach goal before timeout	Gem detours trade off time	Restricted trace; rich audit summaries	Density; budget; deceptive placement

Table 1: Compact summary of the three benchmark environments used in the workshop paper.

Generalization and degeneracy. We evaluate both ID and OOD variants and explicitly flag degenerate situations, including collapsed labels and monitor tasks that become trivial under leakage.

3.4 Leakage-Aware Monitoring

Monitor evaluation is the most fragile part of this benchmark. Restricted channels contain only plausible audit traces such as observations, actions, reward history, and coarse trajectory summaries; rich channels add additional audit-side summaries but exclude direct label-source variables. Before any monitor result is accepted, the repository runs leakage diagnostics that try to predict the target from suspicious individual features or source variables. A monitor result is reported only if the feature set passes that screening.

4 Experimental Setup

We evaluate three symbolic tasks, two policy families, and deterministic ID/OOD variants under a fixed local-compute budget. Each environment is trained with three random seeds. ProxyFarming and SensorTamper use 200,000 PPO steps per run and the recalibrated GemTimeout uses 60,000. Oracle True PPO is included only as a diagnosis baseline for the hidden objective, not as a deployable method.

Monitor evaluation is stricter than the policy benchmark. Monitor datasets are built from stochastic rollouts of the proxy-trained policy family only, and restricted versus rich audit channels are evaluated with logistic regression and gradient boosting under run-aware splits. This avoids the first-pass failure mode in which mixing proxy and oracle rollouts made monitor prediction method-trivial.

5 Results

Specification gaps are measurable across the suite. Figure 2 and Table 2 show large and interpretable gaps between proxy optimization and the hidden objective. In ProxyFarming, Proxy PPO reaches ID true score -1.950 with hack incidence 1.000, while Oracle True PPO reaches 0.992 with zero hack incidence. SensorTamper is sharper still: Proxy PPO reaches ID true score -3.346 with hack incidence 0.833, versus 1.000 and zero for the oracle. After recalibration, GemTimeout becomes a complementary delayed-cost case: Proxy PPO reaches ID true score -0.153 with hack incidence 1.000, while the oracle reaches 1.000 with zero hack incidence. Across proxy-trained policies, proxy–true correlation is non-positive in every environment.

Tampering is explicitly measurable. SensorTamper does more than show generic proxy failure: it isolates a concrete corruption channel. The proxy policy uses the tamper mechanism in 0.417 of evaluation episodes, whereas the oracle never tampers. Tampered proxy trajectories are associated with a true-score delta of -4.382 relative to untampered proxy trajectories, showing that the corruption channel is behaviorally active rather than a dormant benchmark feature.

Leakage-aware monitor evaluation matters. Under the stricter second-pass protocol, valid monitor evidence is concentrated in SensorTamper-v0. A restricted logistic-regression monitor reaches ROC-AUC 0.936 and PR-AUC 0.962; the corresponding rich-channel model reaches 0.964 and 0.986. That gain is real but small, which is scientifically useful: the task is not trivialized, yet richer audit information helps. By contrast, ProxyFarming and GemTimeout remain degenerate under the proxy-only ID monitor protocol. That negative result is part of the paper's argument rather than an

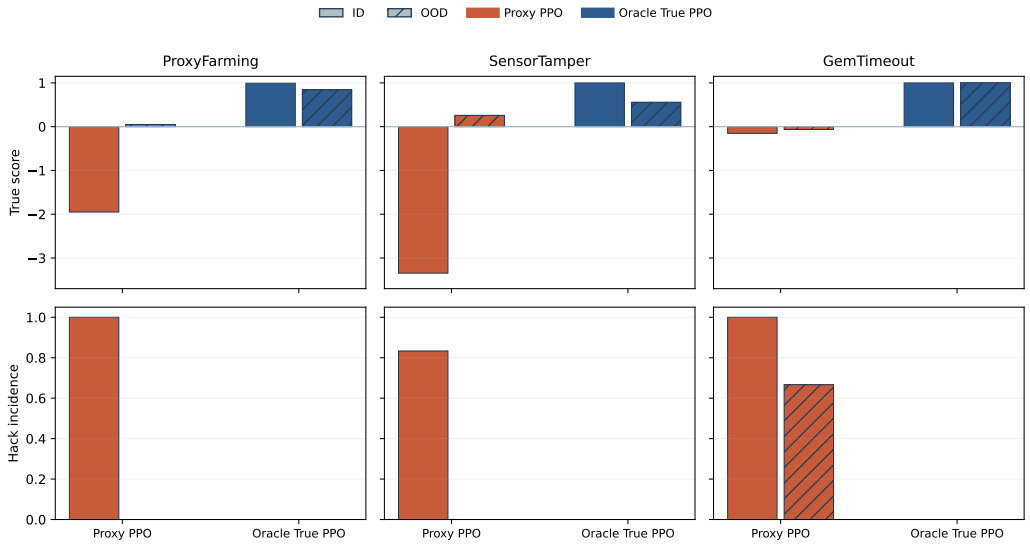


Figure 2: Main benchmark outcomes. Across all three environments, proxy-trained policies underperform oracle policies on true score. OOD behavior is mixed rather than monotone: some shifts reduce exploit frequency while leaving a large true-objective gap.

Environment	Policy	ID true	OOD true	ID hack	OOD hack
ProxyFarming	Proxy PPO	-1.950	0.050	1.000	0.000
	Oracle True PPO	0.992	0.842	0.000	0.000
SensorTamper	Proxy PPO	-3.346	0.258	0.833	0.000
	Oracle True PPO	1.000	0.558	0.000	0.000
GemTimeout	Proxy PPO	-0.153	-0.065	1.000	0.667
	Oracle True PPO	1.000	1.000	0.000	0.000

Table 2: Main specification-gap results. Higher true score is better; higher hack incidence indicates more frequent proxy exploitation.

exception to hide: monitorability evaluation can easily look stronger than it is unless leakage and label degeneracy are treated as first-class diagnostics.

Why this matters for a workshop audience. The benchmark's value is not that it solves monitoring. It is that it makes a failure mode visible: naive monitor evaluation can collapse under leakage or label structure even in small symbolic environments. SpecLens-RLtherefore supports two useful diagnoses at once: it measures proxy and tampering failures, and it exposes when a monitoring story is not yet trustworthy.

6 Limitations

SpecLens-RLis a measurement benchmark, not a realistic deployment simulator. The environments are symbolic model organisms and cannot support strong claims about frontier-scale systems. Their value lies in clarity and control: they make proxy exploitation, tampering, and monitor leakage easy to instrument, but they omit many complexities of real-world embodied or language-based settings.

Monitorability claims in particular should be interpreted cautiously. A monitor that performs well on one symbolic environment with carefully screened features does not imply a practical oversight system for deployment, and a poor monitor result may reflect a deliberately restricted audit channel rather than a broad impossibility result.

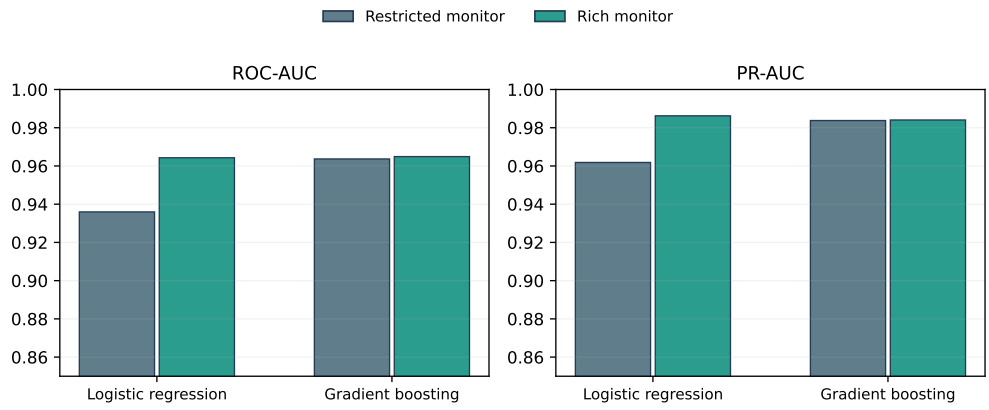


Figure 3: Leakage-cleared hack-monitor results for SensorTamper-v0. Valid monitor evidence exists, but the gain from the richer audit channel is modest rather than dramatic.

Channel	Monitor	ROC-AUC	PR-AUC	Leakage
Restricted	Gradient boosting	0.964	0.984	Pass
Restricted	Logistic regression	0.936	0.962	Pass
Rich	Gradient boosting	0.965	0.984	Pass
Rich	Logistic regression	0.964	0.986	Pass

Table 3: Leakage-cleared hack-monitor results for SensorTamper-v0 under the proxy-only monitor protocol.

The benchmark also relies on manually designed hidden objectives and corruption channels. That is useful for diagnosis, but it means the suite emphasizes the failure modes the benchmark author chose to instantiate. Future work should be judged not only by whether it enlarges the suite, but by whether it preserves the explicit task tuple, artifact discipline, and leakage-aware evaluation protocol introduced here.

7 Conclusion

We presented SpecLens-RL, a compact framework for diagnosing specification failures in reinforcement learning. The core contribution is a reusable task tuple and evaluation protocol that separate proxy reward, hidden objective, corruption channels, monitor channels, distribution shift, and policy family. The benchmark instantiates that tuple in three symbolic environments and ties analysis outputs directly to paper artifacts through a reproducible local pipeline.

The strongest empirical claim is conservative and useful. Across three symbolic tasks, SpecLens-RLexposes large specification gaps and explicit tampering risk, while leakage-cleared monitor evidence is feasible but narrow, with accepted results concentrated in SensorTamper-v0. For workshop purposes, that is a useful methodological message: benchmark leakage and label degeneracy are not side issues, but part of what a trustworthy diagnostic benchmark must surface.

References

Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety, 2016. URL https://arxiv.org/abs/1606.06565.

Stuart Armstrong, Jan Leike, Laurent Orseau, and Shane Legg. Pitfalls of learning a reward function online. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*, pages 1592–1600, 2020. doi: 10.24963/ijcai.2020/221. URL https://doi.org/10.24963/ijcai.2020/221.

Karl Cobbe, Chris Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 2048–2056, 2020. URL https://proceedings.mlr.press/v119/cobbe20a.html.

Tom Everitt, Marcus Hutter, Ramana Kumar, and Victoria Krakovna. Reward tampering problems and solutions in reinforcement learning: A causal influence diagram perspective. *Synthese*, 198 (7):6435–6467, 2021. doi: 10.1007/s11229-021-03141-4. URL https://doi.org/10.1007/s11229-021-03141-4.

Jan Leike, Miljan Martic, Victoria Krakovna, Pedro A. Ortega, Tom Everitt, Andrew Lefrancq, Laurent Orseau, and Shane Legg. AI safety gridworlds, 2017. URL https://arxiv.org/abs/1711.09883.

Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking, 2022. URL https://arxiv.org/abs/2209.13085.